

3-Day Python Bootcamp: Beginner → Intermediate Portfolio Sprint

Goal: By the end of Day 3, you'll have 3 real, working, well-documented projects on GitHub, git/testing habits that look professional to a hiring manager, and a clear roadmap for the weeks after this sprint (because landing a job takes sustained effort — this gets you the ammunition for it).

Format each day: Learn (2-3 hrs) → Build (4-5 hrs) → Ship (30-60 min: commit, README, polish)

Setup once, before Day 1:

```
bash

python --version    # need 3.10+
mkdir portfolio-sprint && cd portfolio-sprint
python -m venv venv
source venv/bin/activate    # venv\Scripts\activate on Windows
git init
```

Create a GitHub account if you don't have one, and a personal access token or SSH key set up for pushing.

Day 1 — Foundations + CLI Automation Tool

Learn (morning)

- Core syntax refresher: variables, loops, conditionals, functions, list/dict comprehensions
- File I/O (`open`), (`with`), reading/writing CSV via the (`csv`) module)
- Exception handling (`try/except/finally`), raising custom exceptions)
- (`argparse`) for building command-line interfaces
- Virtual environments and (`pip freeze > requirements.txt`)

Good free resources: [Automate the Boring Stuff](#) (chapters on files + CLI), Python official docs on (`argparse`).

Build: Project 1 — Bulk File Organizer / Expense Tracker CLI

Pick one (or do both if you have time — they reuse the same skills):

Option A: File Organizer A CLI tool that scans a folder and sorts files into subfolders by type (images, docs, code, etc.), with a `--dry-run` flag, logging, and undo capability (keep a log of moves so they can be reversed).

Option B: Expense Tracker A CLI tool (`expense add`, `expense list`, `expense summary`) that stores entries in a CSV or SQLite file, supports categories, and prints a monthly summary table.

Must-haves for either (this is what makes it "intermediate" not "tutorial"):

- Proper error handling (bad input doesn't crash it)
- Command-line arguments via `argparse`, not hardcoded values
- At least one custom exception class
- Logging via the `logging` module instead of `print` for status/errors

Ship

- Write a README with: what it does, install/run instructions, a GIF or screenshot of it running, and "future improvements"
 - `git add . && git commit -m "..." && git push`
 - Use meaningful, incremental commits (not one giant commit) — this matters more than people realize when recruiters skim your commit history
-

Day 2 — OOP + APIs + Data Project

Learn (morning)

- OOP: classes, `__init__`, inheritance, dunder methods (`__str__`, `__repr__`), when to use a class vs. a dict/function
- Making HTTP requests with `requests`, handling JSON responses
- Reading a public API's docs and handling rate limits / errors gracefully
- Basics of `sqlite3` for local data persistence
- Intro to `pytest`: writing your first unit tests

Build: Project 2 — API-Driven Data Tool

Pick a public API that interests you (no key needed ones: Open-Meteo for weather, PokeAPI, NASA APIs, JSONPlaceholder, or GitHub's own API to build a tool that inspects repos). Build an OOP-structured app, for example:

Weather/Trip Dashboard A class-based app where a `WeatherClient` fetches data, a `Forecast` object represents parsed results, and a small CLI or simple script lets a user check multiple cities, save favorites to SQLite, and see a "should I bring an umbrella" recommendation.

Must-haves:

- At least 2 classes with a clear single responsibility each
- API errors handled (timeout, bad response, no internet) without crashing
- Data persisted locally (SQLite or JSON)
- At least 3-5 `pytest` unit tests (mock the API call so tests don't depend on network — use `unittest.mock`)

Ship

- README with architecture notes: why you structured the classes this way (this is exactly what interviewers ask about)
 - Add a `tests/` folder, show the tests passing
 - Push with clean commit history
-

Day 3 — Web Backend + Capstone Project

Learn (morning)

- REST API basics: what GET/POST/PUT/DELETE mean, status codes
- FastAPI basics (recommended over Flask for 2026 job-market relevance — async support, auto docs, type hints): routes, request/response models with Pydantic
- Connecting FastAPI to SQLite (or SQLAlchemy if you want to push further)
- Deploying for free (Render, Railway, or Fly.io) so you have a **live link**, not just code

Build: Project 3 (Capstone) — Task Manager / Job Application Tracker API

A REST API with full CRUD:

- `POST /tasks` create, `GET /tasks` list, `GET /tasks/{id}` detail, `PUT /tasks/{id}` update, `DELETE /tasks/{id}` delete
- Data validated with Pydantic models
- Persisted in SQLite
- Auto-generated docs via FastAPI's `/docs` (this alone is a great demo to show interviewers)

Stretch goals if time allows:

- Add simple auth (API key check)
- Add filtering/sorting query params ((GET /tasks?status=done))
- Deploy it live and put the URL in your README

Must-haves:

- Clear folder structure ((app/), (models.py), (main.py), (tests/))
- At least a few endpoint tests using FastAPI's (TestClient)
- A (requirements.txt) and simple instructions to run locally

Ship (this is the most important hour of the whole sprint)

- Polished README for all 3 repos: what it does, tech used, how to run it, screenshots/GIFs, live link if deployed
 - Pin these 3 repos on your GitHub profile
 - Write/update a **GitHub profile README** (the special repo named exactly your username) summarizing who you are and linking your top projects
 - Update your resume: for each project, write one impact-style bullet, e.g. *"Built a REST API (FastAPI + SQLite) with full CRUD and test coverage, deployed live"* rather than *"made a task app"*
-

Portfolio Checklist (apply to all 3 repos)

- ☐ README explains what/why/how, not just what
- ☐ Clear commit history (5+ meaningful commits, not one dump)
- ☐ (.gitignore) (no (venv/), (__pycache__/, (.env) committed)
- ☐ At least basic tests present and passing
- ☐ No hardcoded secrets/API keys (use (.env) + (python-dotenv))
- ☐ Consistent code style (consider running (black) once before final commit)

After the 3 Days: The Realistic Roadmap to a Job

Being honest — this is where the actual job-landing work happens:

- **Weeks 1-2:** Deepen one of these projects further (add auth, a frontend, or deploy properly). Depth beats breadth for interviews.

- **Weeks 2-4:** Practice data structures/algorithms (start with easy LeetCode/NeetCode problems) since many screens still test this even for junior roles.
- **Ongoing:** Apply consistently, tailor resume bullets per posting, and use these projects as talking points — be ready to explain *design decisions*, not just what the code does.
- Consider contributing a small PR to an open-source project; it's a strong, differentiating portfolio line.

Good luck — you'll come out of these 3 days with something real to show, which already puts you ahead of most "I did some tutorials" applicants.